



*Internship at
Center for Augmented and Virtual Environment Lab*

“ 3D Evidence Reconstruction on Android using 2D Videos ”

Submitted by

Inchara K Kuppal

PES1UG25CS818

8 Weeks Summer Internship

Under the guidance of

Dr. Adithya Balasubramanyam

Professor

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY**

(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

DECLARATION

I hereby declare that the project entitled “3D Evidence Reconstruction using 2D Videos” has been carried out at *Center for Augmented and Virtual Environment Lab* by me under the guidance of Dr. Adithya Balasubramanyam, Professor and submitted in partial fulfillment of the credits for the degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester 4 Summer Internship for 8 weeks. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES1UG25CS818

INCHARA K KUPPAL

ACKNOWLEDGEMENT

I would like to express my gratitude to my guide **Dr. Adithya Balasubramanyam, Professor**, for their continuous guidance, assistance and encouragement throughout the development of this project.

I am grateful to the internship coordinators **Prof. Mahitha G** and **Dr. Bhargavi M**, Dept. of Computer Science and Engineering, PES University for organizing, managing and helping out with the entire process.

I take this opportunity to thank **Dr. Shylaja S S, Chairperson**, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to **Prof. Jawahar Doreswamy, Chancellor**, PES University, and **Dr. Suryaprasad J, Vice-Chancellor**, PES University for providing me various opportunities and enlightenment every step of the way.

I also wish to extend my sincere thanks to **Manoj Kumar H R**, who instructed and guided us on the right path throughout this Internship.

Finally, this internship could not have been completed without the continual support and encouragement I have received from my parents and my friends.

ABSTRACT

The Preservation of forensic crime scenes traditionally relies on specialized LiDAR equipment or multi-camera rigs, which can be resource-intensive and inaccessible in time-sensitive scenarios. To address this limitation, this project presents an Android app SplatStudio, an end-to-end pipeline designed to transform standard monocular walkthrough videos into navigable 3D Gaussian Splat representations. Developed with an explicit focus on forensic documentation, the system enables investigators to digitally preserve physical spaces using standard mobile recording devices, allowing for subsequent comprehensive review even after a scene has been physically altered or released.

The technical architecture is built upon a FastAPI backend and operates through a robust three-stage processing chain. Initially, uploaded video files are decomposed into a frame sequence utilizing FFmpeg, sampled at approximately two frames per second to optimize the balance between spatial coverage and computational redundancy. Subsequently, the extracted frames are processed using COLMAP for classical Structure-from-Motion (SfM). This stage executes feature extraction, exhaustive matching, and incremental sparse mapping to compute accurate camera poses and generate a foundational sparse point cloud. In the final stage, this geometric data is processed via a Gaussian splatting module to synthesize a dense, photorealistic 3D representation. The resulting output is exported as a .ply point cloud and served securely via an API endpoint.

Unlike generic video-to-3D models optimized solely for visual fidelity, the architectural roadmap of SplatStudio prioritizes evidentiary reliability. While the core capture-to-splat pipeline is currently fully functional, active development focuses on forensic-grade enhancements. Future implementations include integrating semantic, object-aware segmentation for isolating evidence, implementing comprehensive audit logging for procedural traceability, applying adaptive statistical outlier rejection to prevent the inadvertent loss of genuine scene data, and introducing metric-scale calibration to ensure dimensional trustworthiness. Ultimately, these advancements establish a highly accessible and scalable framework for modern forensic scene analysis.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Overview of the Research Project Context	
1.2 Vision and Mission	
1.3 Target Application: Forensic Crime-Scene Documentation	
 CHAPTER 2: PROJECT OVERVIEW & ARCHITECTURE	 2
2.1 Problem Statement & Objectives	
2.2 System Architecture: Decoupling Frontend and Backend	
2.3 Scope, Limitations, and Hardware Constraints	
2.4 My Roles and Responsibilities	
 CHAPTER 3: SYSTEM ARCHITECTURE & FOUNDATIONAL PIPELINE	 4
3.1 Monocular-Video-to-3D Workflow	
3.2 Frame Extraction and Sampling Trade-offs (FFmpeg)	
3.3 Structure-from-Motion Integration (COLMAP)	
3.4 Gaussian Splatting Training Invocation (FastGS)	
3.5 Pipeline Stabilization and Fault Tolerance	

CHAPTER 4: CORE IMPLEMENTATION & FORENSIC ENHANCEMENTS	7
4.1 Statistical Point-Cloud Filtering	
4.1.1 Scale and Anisotropy Filtering	
4.1.2 Adaptive Opacity Filtering (MAD-based Thresholding)	
4.1.3 Density-Based Clustering (Adaptive DBSCAN)	
4.2 Appearance-Based Filtering for Artifact Removal	
4.3 Object Segmentation and Ground-Plane Removal	
4.4 Forensic Audit Trail Implementation	
4.5 Backend API and Job-State Orchestration	
4.6 Cross-Platform Deployment Automation	
CHAPTER 5: FRONTEND APPLICATION (SPLATSTUDIO)	12
5.1 Mobile Client Architecture	
5.2 API Communication and Job Status Tracking	
5.3 Navigating the 3D Reconstructed Environment	
CHAPTER 6: EVALUATION & CALIBRATION METRICS	14
6.1 Metric-Scale Calibration via ArUco Markers	
6.2 COLMAP Reliability Parsing and Defensibility Masks	
6.3 Render-Quality Metrics (PSNR, SSIM, LPIPS)	
CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS	16
7.1 Summary of Project Outcomes	
7.2 Future Scope	
REFERENCES	17

LIST OF FIGURES

Figure 3.1: High-level Architecture of the SplatStudio Pipeline	4
Figure 3.2: Structure-from-Motion (SfM) Sequential Matching Workflow	5
Figure 4.1: Audit Trail JSON / Logs	10
Figure 5.1: SplatStudio Dashboard/Upload and Progress Bar	12
Figure 5.2: 3D Gaussian Splat Rendered on SplatStudio Mobile Interface	13
Figure 6.1: Quality Matrices on SplatStudio Mobile Interface	15

LIST OF TABLES

Table 3.1: FFmpeg and COLMAP Configuration Parameters	6
Table 4.1: Scale and Anisotropy Filtering Criteria	7

CHAPTER 1: INTRODUCTION

1.1 Overview of the Research Project Context

The rapid evolution of computer vision and spatial computing has fundamentally shifted how physical environments are digitized. This project, "3D Evidence Reconstruction on Android using 2D Videos," was carried out at the Center for Augmented and Virtual Environment (CAVE) Lab. The research conducted here focuses on bridging the gap between highly specialized spatial capture hardware and accessible, consumer-grade recording devices by engineering a pipeline capable of processing monocular video inputs into dense, navigable 3D representations.



1.2 Vision and Mission

Vision: To democratize 3D spatial capture by replacing expensive, specialized hardware dependencies with highly accessible, software-driven mobile technologies, making high-fidelity spatial preservation instantly available via standard smartphone cameras.

Mission: To develop robust, computationally efficient computer vision pipelines that transform ordinary 2D video sequences into mathematically defensible 3D environments.

1.3 Target Application: Forensic Crime-Scene Documentation

The primary application driving the architectural design of this project is forensic crime-scene documentation. Traditionally, scene preservation relies on specialized LiDAR scanners or complex multi-camera rigs. These tools, while highly accurate, are exceptionally resource-intensive and often unavailable during time-sensitive, initial investigations. Unlike generic video-to-3D pipelines that optimize purely for gaming assets or visual fidelity, SplatStudio prioritizes evidentiary reliability.

CHAPTER 2: PROJECT OVERVIEW & ARCHITECTURE

2.1 Problem Statement & Objectives

Converting unstable, monocular handheld video into mathematically accurate 3D geometry without specialized hardware like LiDAR presents a significant computational challenge. Generic photogrammetry pipelines often produce floating artifacts, visual ghosting, and scale-ambiguous environments that are unusable in a forensic context. The primary objective of this project was to engineer an automated, robust pipeline that extracts frames, computes structure-from-motion (SfM), trains a dense Gaussian Splat, and algorithmically filters out geometry noise to produce a defensible, highly accurate 3D point cloud for evidentiary review.

2.2 System Architecture: Decoupling Frontend and Backend

To achieve mobile accessibility without sacrificing processing power, the system architecture was strictly decoupled into client and server entities. **SplatStudio** functions as the Android mobile application, acting as the client-side interface for investigators to capture scene videos and visualize the final 3D environments. Conversely, the FastAPI Python backend operates as the heavy-compute server. This client-server model allows lightweight mobile devices to initiate and poll heavyweight GPU computations, effectively offloading the resource-intensive tasks of FFmpeg video processing, COLMAP mapping, and FastGS training to a dedicated machine.

2.3 Scope, Limitations, and Hardware Constraints

The scope of this project encompasses the end-to-end processing of a 2D video file into a cleaned, navigable .ply Gaussian Splat, alongside the API infrastructure required to serve it to a mobile client. However, the system operates under strict hardware constraints. The backend necessitates a dedicated CUDA-enabled GPU to run FastGS and COLMAP feature matching efficiently. Furthermore, to balance SfM feature density against VRAM limitations and processing time, video inputs are programmatically constrained to a 3 frames-per-second (FPS) sampling rate, mitigating redundant, near-duplicate frame extraction.

2.4 My Roles and Responsibilities

As a Research Intern at the Center for Augmented and Virtual Environment (CAVE) Labs, my primary responsibility was to architect and engineer the heavy-compute backend processing pipeline. My role required a deep dive into computer vision algorithms, mathematical filtering, and API infrastructure. My specific contributions and responsibilities over the 8-week internship were structured across the following developmental phases:

- **Weeks 1 - 2 (Foundational Pipeline Development):** Wired the foundational structure-from-motion (SfM) sequence. I configured FFmpeg for optimal frame extraction (3 FPS) and integrated COLMAP's sequential matcher to process the frames into a sparse point cloud. I also integrated the FastGS Gaussian Splatting training loop to generate the initial 3D output.
- **Weeks 2 - 3 (Pipeline Stabilization):** Engineered fault-tolerant orchestration logic to tie the extraction, SfM, and training stages together. I implemented custom execution wrappers to capture subprocess errors, handle headless server constraints, and prevent silent pipeline failures.
- **Weeks 3 - 5 (Statistical Cleanup and Auditing):** Completely rebuilt the point-cloud cleanup pipeline using raw NumPy and SciPy operations. I developed adaptive, MAD-based thresholding for opacity filtering and k-distance elbow methods for density-based DBSCAN clustering. Concurrently, I built an additive, non-invasive forensic audit logging system to record the exact threshold and reason for every point removed.
- **Weeks 4 - 6 (Appearance Filtering and Segmentation):** Authored algorithms to detect visual artifacts by cross-referencing spherical-harmonics color consistency and checking for degenerate rotation quaternions. I implemented RANSAC-based ground-plane removal and object isolation module to separate critical forensic evidence from background clutter.
- **Weeks 6 - 7 (Evaluation and Calibration):** I parsed COLMAP reliability statistics and integrated render-quality metrics like PSNR and SSIM.
- **Weeks 7 - 8 (API Orchestration and Deployment):** Developed the FastAPI routing and job-state orchestration, ensuring secure and stable communication with the SplatStudio mobile app. Finally, I authored cross-platform bootstrap scripts for automated dependency management and deployment across diverse Linux and Windows environments.

CHAPTER 3: SYSTEM ARCHITECTURE & FOUNDATIONAL PIPELINE

3.1 Monocular-Video-to-3D Workflow

The foundation of the SplatStudio backend is a sequential processing chain designed to convert unstructured video into dense 3D geometry. The workflow operates in three primary phases: temporal frame extraction, sparse geometric mapping, and dense photometric reconstruction. This architecture ensures that raw mobile video captured by an investigator is systematically broken down and mathematically rebuilt into a defensible spatial model.

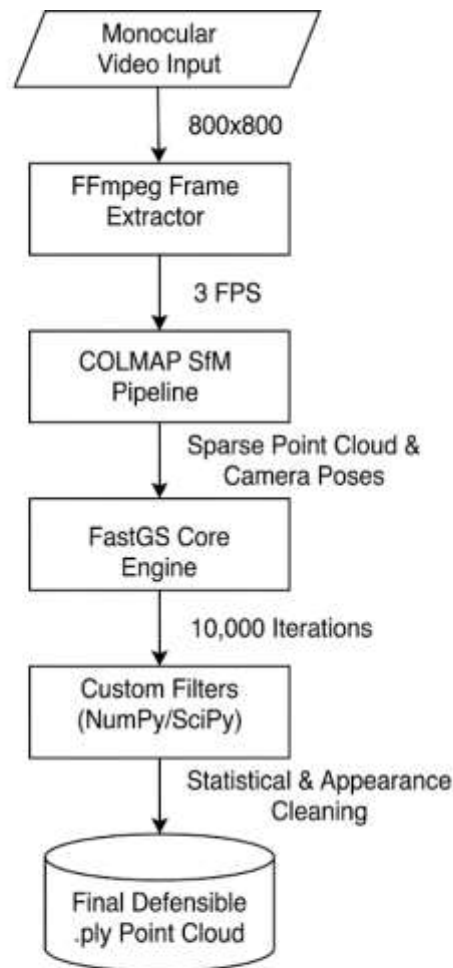


Figure 3.1: High-level Architecture of the SplatStudio Pipeline

3.2 Frame Extraction and Sampling Trade-offs (FFmpeg)

The first stage utilizes FFmpeg to decompose the uploaded video into a discrete frame sequence. The extraction is strictly configured at a sampling rate of 3 frames per second (FPS) with an aspect-preserving resize to an 800x800 resolution. This configuration represents a deliberate engineering trade-off; oversampling frames from a slow, handheld forensic walkthrough introduces redundant, near-duplicate views that exponentially inflate downstream feature-matching costs without contributing new geometry.

3.3 Structure-from-Motion Integration (COLMAP)

Extracted frames are passed to COLMAP to execute a four-stage Structure-from-Motion (SfM) sequence: feature extraction, sequential matching, mapping, and image undistortion. The feature extractor is configured to use SIFT with an OPENCV camera model and affine-shape estimation, capped at 16,384 maximum features per image. Crucially, a sequential matcher is employed rather than exhaustive pairwise matching. Since forensic walkthrough videos inherently possess strong frame-to-frame temporal ordering, exhaustive matching would result in an $O(n^2)$ time complexity, which is highly computationally wasteful for sequential data.



Figure 3.2: Structure-from-Motion (SfM) Sequential Matching Workflow here, center-aligned

3.4 Gaussian Splatting Training Invocation (FastGS)

The sparse geometry and camera poses generated by COLMAP serve as the initialization for the dense reconstruction phase, handled by the FastGS module. The Gaussian Splatting training loop is invoked for 10,000 iterations to synthesize a dense, photorealistic 3D representation. An evaluation flag is explicitly utilized during this invocation to hold out specific validation views, which are critical for later computing render-quality metrics such as PSNR and SSIM.

Module	Parameter	Configuration Value
FFmpeg	Sampling Rate	3 FPS
FFmpeg	Output Resolution	800 × 800 (Aspect-Preserving)
COLMAP	Feature Descriptor	SIFT
COLMAP	Camera Model	OPENCV
COLMAP	Keypoint Limit	16,384 Features
COLMAP	Matcher Type	Sequential Matcher
FastGS	Training Length	10,000 Iterations
FastGS	Evaluation Mode	- - eval Enabled

Table 3.1: FFmpeg and COLMAP Configuration Parameters here, center-aligned

3.5 Pipeline Stabilization and Fault Tolerance

To transition the sequence from a brittle script into a robust server environment, fault-tolerant orchestration logic was engineered. The FFmpeg, COLMAP, FastGS, and cleanup stages are bound within distinct exception boundaries rather than a monolithic block, allowing for precise point-of-failure diagnosis. Furthermore, custom execution wrappers were implemented to capture subprocess output, automatically inject environment fixes (such as resolving MKL/OpenMP threading conflicts or setting headless server displays), and prevent silent execution hangs, ensuring the server could run unattended.

CHAPTER 4: CORE IMPLEMENTATION & FORENSIC ENHANCEMENTS

4.1 Statistical Point-Cloud Filtering

A critical limitation of off-the-shelf Gaussian Splatting pipelines is the generation of spurious geometry - floating artifacts and background noise that compromise the evidentiary integrity of a scene. Initially, an Open3D - based cleanup script was utilized; however, this was entirely rebuilt from scratch into a custom NumPy and SciPy pipeline. By reading and writing raw binary `.ply` data directly, the system achieves granular, attribute-level control over every Gaussian point, ensuring that filtering decisions are mathematically sound rather than arbitrary.

4.1.1 Scale and Anisotropy Filtering

The first filtering pass evaluates the physical dimensions of each Gaussian. Because the training engine stores scale in logarithmic format, the values are first exponentiated to retrieve true scene scales. Gaussians are then evaluated against specific morphological criteria. Points classified as "bloated" (maximum axis exceeding a scene-relative threshold), "needle-shaped" (anisotropy ratio of maximum to minimum axis exceeding 50:1), or "dust" (near-zero scale) are structurally unstable and programmatically stripped from the point cloud.

Filter Category	Mathematical Condition	Forensic Engineering Rationale
Bloated Gaussians	$\text{Max_Scale} > \text{Scene_Extent_Threshold}$	Removes massive, low-opacity background fillers.
Needle-Shaped	$\text{Max_Scale} / \text{Min_Scale} > 50$	Eliminates degenerate, stretched artifacts.
Dust Particles	$\text{Approx Volume} = 0$	Clears microscopic, visually irrelevant noise.

Table 4.1: Scale and Anisotropy Filtering Criteria

4.1.2 Adaptive Opacity Filtering (MAD-based Thresholding)

Standard point-cloud filtering relies on fixed-percentile cutoffs (e.g., dropping the lowest 5% of opacity). In a forensic context, this is fundamentally flawed: a clean scan loses valuable data unnecessarily, while a highly contaminated scan remains noisy. To solve this, a Median Absolute Deviation (MAD) modified Z-score was implemented. Utilizing the standard Iglewicz and Hoaglin scaling factor of 0.6745 and a Z-threshold of 3.5, this adaptive threshold dynamically adjusts to the actual noise distribution of each individual scan, providing robust outlier rejection without inadvertently discarding genuine scene evidence.

4.1.3 Density-Based Clustering (Adaptive DBSCAN)

To isolate primary scene geometry from scattered floating artifacts, DBSCAN clustering is applied. Hardcoding the neighborhood radius (eps) breaks across scenes of varying point densities. Instead, an adaptive k-distance elbow method was engineered. The algorithm calculates each point's distance to its k-th nearest neighbor and plots the distribution curve. A Kneedle-style perpendicular-distance calculation is then applied against the line connecting the curve's endpoints to find the point of maximum curvature. This "elbow" dynamically determines the optimal eps threshold for the specific density of the current reconstruction.

4.2 Appearance-Based Filtering for Artifact Removal: Geometric validations alone are insufficient for complete artifact removal; a point can pass scale, opacity, and density checks but still manifest as a severe visual anomaly. To combat this, orthogonal appearance-based filtering was implemented to ensure both photometric and orientational sanity across the 3D model.

- **Color-Consistency Validations:** The algorithm analyzes the spherical-harmonics DC term (the view-independent base color) of each Gaussian and compares it against the mean color of its k nearest spatial neighbors.
- **Rotation Sanity Checks:** The training engine stores Gaussian orientations as unnormalized quaternions. The filter evaluates the quaternion norm of each point, identifying those that deviate significantly from unit length. A near-degenerate quaternion mathematically signals an optimizer failure, resulting in a meaningless spatial orientation that must be discarded.

4.3 Object Segmentation and Ground-Plane Removal: To make the reconstruction viable for focused forensic analysis, the central evidentiary object must be isolated from environmental clutter. I deliberately engineered a classical spatial heuristic approach for this module rather than integrating a heavy semantic machine-learning model, intentionally avoiding the fragile dependency footprint and GPU overhead of such systems.

- **Verticality-Checked RANSAC:** A single-dominant-plane RANSAC algorithm is applied to detect and strip the floor. Crucially, an explicit verticality check on the fitted normal is enforced; the plane is only removed if its normal is near-vertical, actively preventing the algorithm from accidentally deleting walls or the physical evidence.
- **Centroid-Proximity Isolation:** Following ground-plane removal, adaptive DBSCAN clustering groups the remaining structures. These clusters are then scored based on their centroid's proximity to the median camera position calculated from the investigator's capture path. The closest clusters are preserved as the primary evidence, while a built-in fail - safe ensures any mathematical exception safely returns the unsegmented cloud rather than halting the pipeline.

4.4 Forensic Audit Trail Implementation: For a tool deployed in crime-scene reconstruction, opaque point-cloud filtering is unacceptable. An investigator or defense expert must know exactly what data was removed and by what rule. To guarantee reproducibility, a comprehensive audit logging layer was integrated into the backend.

- **Granular Action Tracking:** Every filtering stage writes through this logging layer, appending the stage name, the exact number of points removed, the precise numeric threshold utilized, and a timestamp into a structured, per-job JSON log.
- **Non-Invasive Architecture:** The logger operates as a strictly additive module. It observes and records decisions without ever altering the filter masks, and it is wrapped in isolated exception handlers ensuring that an I/O logging failure can never crash the actual geometric cleanup process.

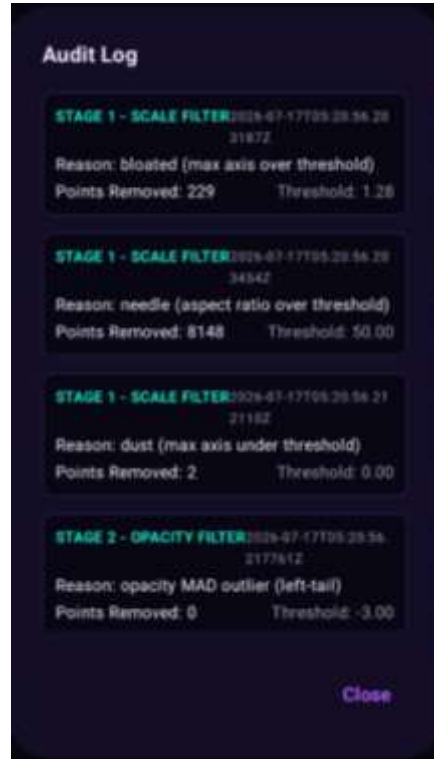


Figure 4.1 - Audit Trail JSON / Logs

4.5 Backend API and Job-State Orchestration

The interaction between the SplatStudio mobile client and the processing server is mediated by a robust FastAPI routing layer. To prevent silent pipeline corruption, the job-tracking infrastructure was built around a self-healing, structured dictionary (containing ID, title, status, progress, and output URLs). A dedicated setter function safely mutates this state across every pipeline stage, strictly preventing stray non-dictionary assignments from crashing the routes. Furthermore, the API exposes granular, human-readable failure states. Instead of returning a generic error, the system specifically identifies if a job failed during FFmpeg extraction, COLMAP tracking (e.g., "insufficient texture overlap"), or FastGS training, allowing the investigator to immediately understand why a capture was rejected.

4.6 Cross-Platform Deployment Automation

Deploying a pipeline with intertwined COLMAP, CUDA, PyTorch, and FastGS dependencies across varying Linux environments is highly prone to version conflicts. To solve this, a comprehensive bootstrap script was authored to automate server instantiation. This script dynamically detects the host operating system and package manager (apt, dnf, pacman, brew), verifies hardware compatibility (Python $\geq$ 3.10, RAM, GPU/VRAM via nvidia-smi), and configures isolated environments for the backend and FastGS separately. The script is idempotent and utilizes environment variables for dynamic pathing, allowing the system to be rapidly deployed or updated on fresh hardware without manual dependency debugging.

CHAPTER 5: FRONTEND APPLICATION (SPLATSTUDIO)

5.1 Mobile Client Architecture

SplatStudio serves as the Android-based interface designed explicitly for field investigators. Operating a complex 3D reconstruction pipeline requires an intuitive, zero-friction frontend that abstracts the heavy compute processes away from the user. The application allows investigators to record standard walkthrough videos of a crime scene and directly upload them to the backend server in a single workflow. By handling video compression and network packaging on-device, the app ensures that large data payloads are transmitted efficiently even over unstable field networks.

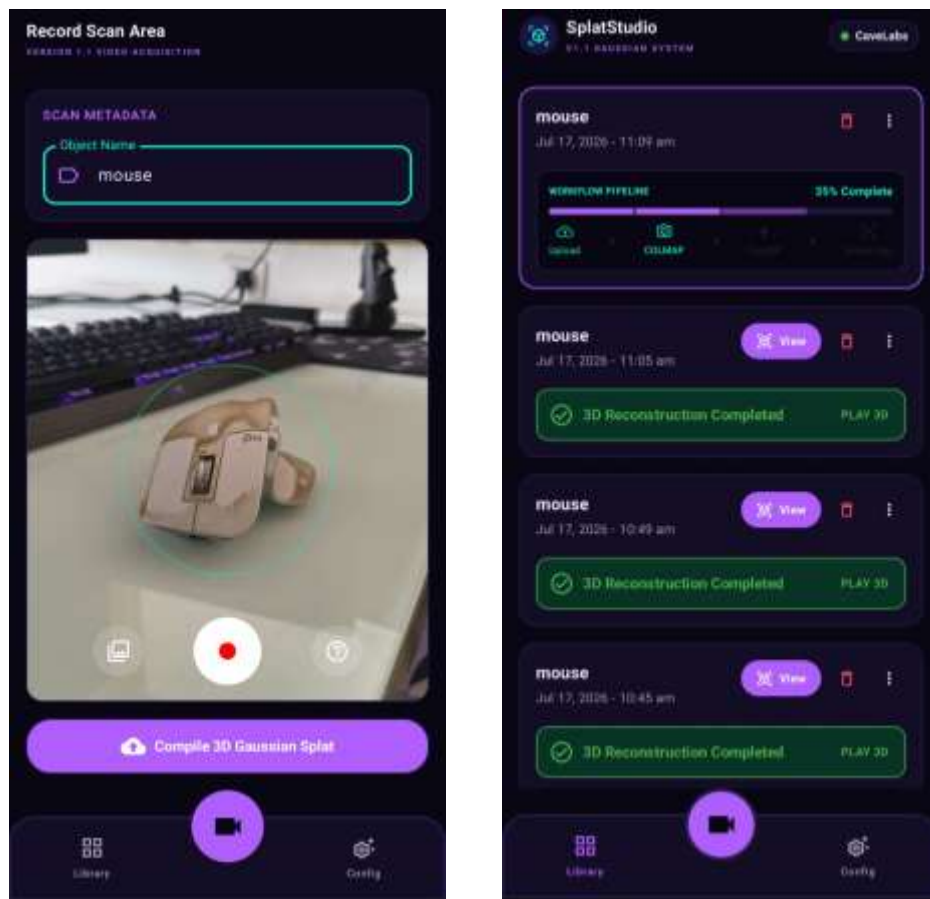


Figure 5.1 - SplatStudio Dashboard / Upload and Progress Bar

5.2 API Communication and Job Status Tracking

Once a video is securely uploaded, the mobile client enters a long-polling communication state with the backend API. SplatStudio continuously queries the server's job-state endpoints to retrieve real-time progress updates. The UI dynamically reflects the active stage of the backend pipeline—Extraction, SfM Mapping, Training, or Cleanup. This transparent communication layer ensures that investigators are not left blindly waiting for results and can retrieve the granular error logs generated by the backend if a capture fails to reconstruct.

5.3 Navigating the 3D Reconstructed Environment

Upon successful completion of the backend processing, SplatStudio securely downloads the filtered .ply point cloud to the local Android device. Rendering a dense 3D Gaussian Splat on mobile hardware requires optimized memory management and efficient rasterization techniques. The application features an intuitive touch-based navigation system, allowing investigators to pinch-to-zoom, pan, and dynamically orbit around the central forensic evidence. This on-device rendering capability is critical; it enables officers to visually cross-reference the digital twin against field notes or physical layouts immediately, without requiring a desktop workstation.



Figure 5.2: 3D Gaussian Splat Rendered on SplatStudio Mobile Interface

CHAPTER 6: EVALUATION & CALIBRATION METRICS

6.1 Metric-Scale Calibration via ArUco Markers

A fundamental limitation of monocular Structure-from-Motion and Gaussian Splatting is scale ambiguity - the resulting 3D geometry has no inherent real-world units. For forensic applications, this is unacceptable, as investigators must be able to measure exact distances between pieces of evidence. To resolve this, I engineered a physical scale calibration module. The algorithm detects a printed ArUco marker (specifically DICT_4X4_50) of known real-world dimensions across multiple video frames. By extracting the marker's corners and applying Direct Linear Transformation (DLT) triangulation using COLMAP's recovered camera poses, the system derives a precise millimeters-per-scene-unit conversion factor. Crucially, if the marker is not detected in at least two frames, the module strictly returns a null value rather than fabricating a scale, adhering to the forensic principle that a measurement tool must never guess.

6.2 COLMAP Reliability Parsing and Defensibility Masks

While Gaussian Splatting produces visually dense environments, it cannot inherently distinguish between highly corroborated geometry and extrapolated visual noise. To establish geometric defensibility, the pipeline parses COLMAP's sparse reconstruction output - specifically analyzing point IDs, XYZ coordinates, and per-point observation track lengths. The algorithm identifies which 3D points are strongly triangulated by multiple camera views versus those weakly observed by only one or two frames. A KD-tree-based spatial comparison is then executed between the dense Gaussian output and these reliable sparse points. This generates a per-Gaussian defensibility mask, visually highlighting regions of the reconstruction that are mathematically certain versus those that are interpolated.

6.3 Render-Quality Metrics (PSNR, SSIM, LPIPS)

To quantify the visual fidelity of the final reconstruction, the backend computes standard image quality metrics on the held-out validation views generated during the FastGS --eval phase. Peak Signal-to-Noise Ratio (PSNR) evaluates the absolute pixel-level error, while the Structural Similarity Index Measure (SSIM) assesses the perceived structural integrity and luminance against the ground-truth video frames. Additionally, where dependency parameters allow, Learned Perceptual Image Patch Similarity (LPIPS) is computed to measure high-level feature similarity. Together, these metrics provide a mathematical defense of how faithfully the generated 3D splat represents the original captured scene.

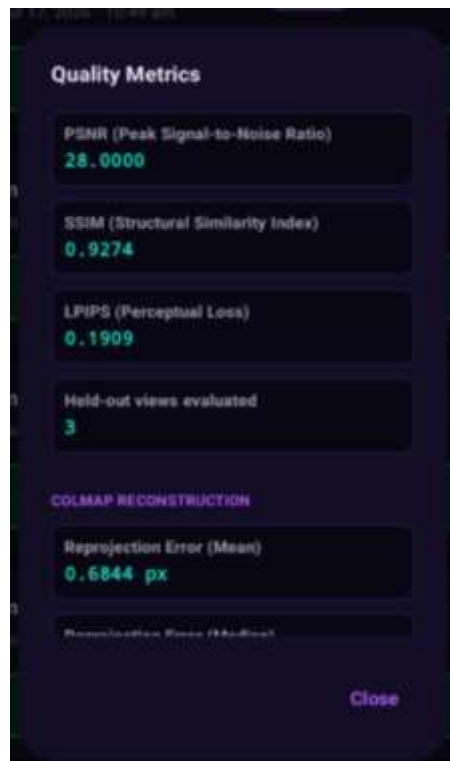


Figure 6.1: Quality Matrices on SplatStudio Mobile Interface

CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS

7.1 Summary of Project Outcomes

The development of SplatStudio during this 8-week internship at the Center for Augmented and Virtual Environment (CAVE) Lab successfully established a novel, end-to-end framework for 3D evidence reconstruction. By rigorously decoupling a lightweight Android frontend from a heavy-compute FastAPI Python backend, the project proved that specialized LiDAR hardware is no longer a strict prerequisite for spatial forensic documentation. The custom-engineered statistical and appearance-based point-cloud filters effectively transformed the raw, often noisy output of Gaussian Splatting into a mathematically defensible, clean .ply point cloud. Furthermore, the integration of physical-scale calibration and strict forensic audit trails ensures that the generated 3D environments are not merely visual novelties, but structured, reliable digital twins ready for rigorous investigative analysis.

7.2 Future Scope

While the foundational pipeline is functionally robust, there are several avenues for future enhancement to elevate the system to enterprise-grade forensic standards:

- **Semantic Object Segmentation:** While current object isolation relies on spatial heuristics (RANSAC and DBSCAN), future iterations could integrate lightweight visual foundation models (e.g., Segment Anything) to provide semantic, object-aware tagging of specific evidence without requiring manual threshold tuning.
- **Dynamic Threshold Optimization:** The adaptive MAD-based filtering could be further augmented with a machine-learning feedback loop, allowing the backend to dynamically adjust geometric outlier thresholds based on lighting and texture conditions extracted from the initial video frames.
- **Enhanced On-Device Rendering:** Further optimization of the Android viewer is planned, including implementing level-of-detail (LOD) degradation and tiled rendering, to support smooth navigation of dense splats on lower-end mobile devices.

REFERENCES

1. Kerbl, B, Kopanas, G, Leal-Taixé, L, & Drettakis, G. (2023). "3D Gaussian Splatting for Real-Time Radiance Field Rendering". *ACM Transactions on Graphics*, 42(4).
2. Schönberger, J. L., & Frahm, J.M. (2016). "Structure-from-Motion Revisited". *Conference on Computer Vision and Pattern Recognition (CVPR)*. (COLMAP framework).
3. Ester, M., Kriegel, H.P, Sander, J, & Xu, X. (1996). "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise" (DBSCAN). *KDD*.
4. Iglewicz, B., & Hoaglin, D. C. (1993). *How to Detect and Handle Outliers*. ASQC Quality Press. (MAD-based thresholding algorithms).
5. FFmpeg Developers. (2023). *FFmpeg Tool (Version 6.0)*. Retrieved from
6. FastAPI Framework Documentation. (2023).
7. Clean – GS: Semantic Mask – Guided Pruning for 3D Gaussian Splatting from Subhankar Mishra.